Linked List - 3

Agenda:

1. Merge Two Sorted Arrays
2. Merge Sort on linked list
3. Check if given linked list has cycle.
4. Find the start of the cycle

① Finding the Middle Element

- **Approach: 1 (Two-Pass):** First, Traverse the entire list to find the length N. Then, traverse again for $(N-1)/2$ nodes to reach the middle.
- **Approach: 2 (Slow & fast Pointers):**

Code →

```
public Node getMiddle (Node head){
        if (head = null) return null;
        Node slow = head;
        Node fast = head;
        while (fast.next != null && fast.next.next != null)
                slow = slow.next;
                fast = fast.next.next;
        }
        return slow;
}
```

② Merge Two Sorted List

```
public Node merge (Node h1, Node h2){
    if (h1 == null) return h2;
    if (h2 == null) return h1;

    Node temp, head;
    if (h1.data <= h2.data) {
            head = h1
            h1 = h1.next;
    }
    else {
            head = h2;
            h2 = h2.next;
    }
    temp = temp.next;
}
if (h1 != null) temp.next = A;
    else temp.next = B;
return head;
}
}
```

$\boxed{T.C = O(N+M)}$

③ Merge Sort on Linked List

```
Public Node sort (Node head){
    if (head== null || head.next ==null) return head;

    Node mid =getMiddle (head);
    Node h2 = mid.next;
    mid.next = null ;   //Split the list

    Node left = sort(head);
    Node right = sort(h2);

    return merge(left,right);
}
```

④ Cycle Detection (Floyd's Cycle-Finding Algorithm)

A cycle exists if a node's next pointer points backs to a previous node in the list.

- Logic: Use slow and fast pointers. If they ever meet (i.e., slow==fast), a cycle exists. If fast reaches null, there is no cycle.

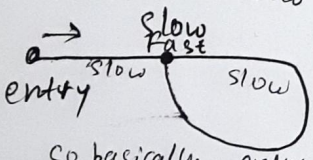
```
Public boolean hasCycle (Node head){
    Node slow =head;
    Node fast = head;
    while (fast != null && fast.next != null){
        slow = slow.next;
        fast = fast.next.next;
        if (slow == fast) return true; //cycle detected
    }
    return false;   //No cycle.
}
```

5 Find the Start of the Cycle

```
public Node findCycleStart (Node head){
    Node slow = head;
    Node fast = head;
    while (fast != null && fast.next != null){
        slow = slow.next;
        fast = fast.next.next;
        if (slow == fast){
            Node entry = head;
            while (entry != slow){
                entry = entry.next;
                slow = slow.next;
            }
            return entry;
        }
    }
    return null;
}
```

fast = 2× slow

fast = slow + slow



So basically, entry = $\frac{fast}{2}$ = slow